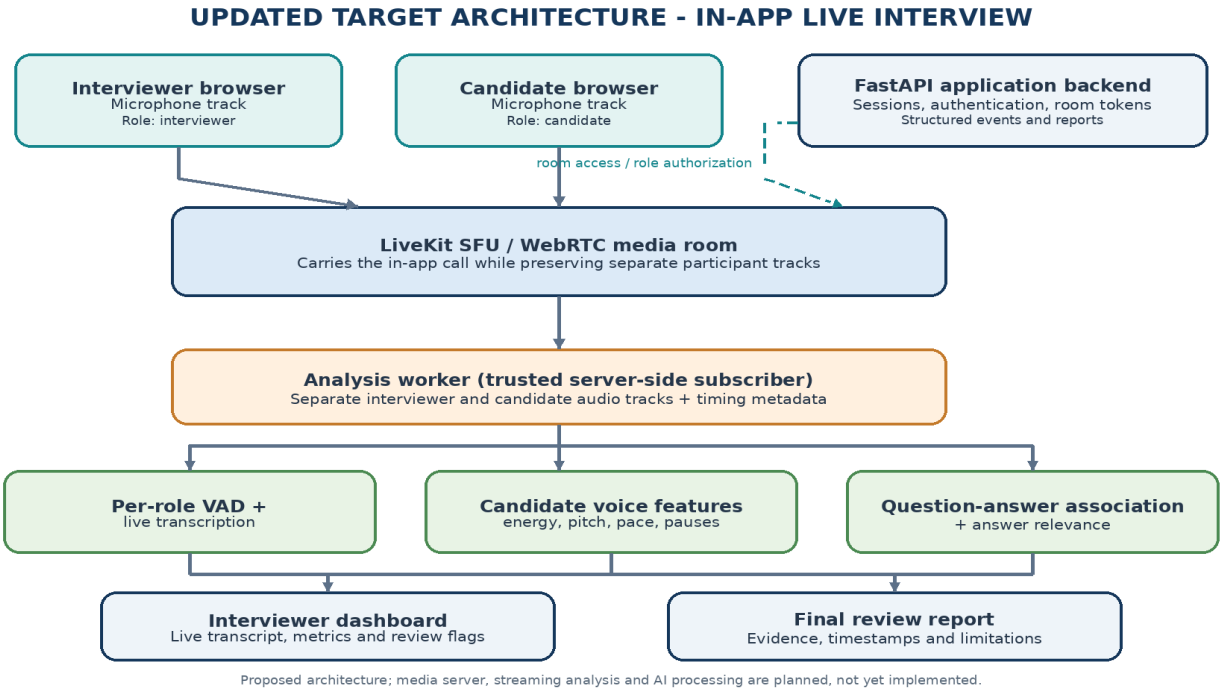
VOICE INTERVIEW ANALYSIS SYSTEM

Project Documentation and Implementation Guide

Live English interview analysis for an in-app call across two networked computers



Document version: 1.1 | Status: Updated target architecture; in-app calling and AI analysis remain planned

Prepared for internal engineering review

Document Control

Field	Value
Project name	Voice Interview Analysis System
Document type	Software project documentation / implementation guide
Version	1.1
Target users	Interviewers, recruiters, and authorized reviewers
Language scope	English
Input configuration	One microphone per participant on separate computers; in-app WebRTC call
Primary interaction	Live interview monitoring through a web dashboard
Current implementation status	Current starter: local microphone-test prototype. Planned: in-app WebRTC call, per-track server analysis, transcription, dashboard events, and reports.

Document Purpose

This document describes the intended Voice Interview Analysis System, including the updated two-computer in-app calling architecture, functional requirements, implementation stages, starter prototype, validation strategy, and known limitations. It distinguishes the existing local microphone-test prototype from the planned WebRTC call, server-side media analysis, and AI features.

Contents

- 1. Executive Summary and Problem Statement
- 2. Goals, Scope, and Non-Goals
- 3. Requirements and Acceptance Criteria
- 4. System Architecture and Data Flow
- 5. Functional Modules
- 6. Technology Stack
- 7. Repository Structure and Setup
- 8. Initial Prototype: How It Works
- 9. Planned API and Event Contracts
- 10. Testing and Evaluation Plan
- 11. Security, Privacy, and Responsible Use
- 12. Known Limitations and Risks
- 13. Development Roadmap
- 14. Glossary and References

1. Executive Summary and Problem Statement

The Voice Interview Analysis System is a live English-language interview assistance web application. The interviewer and candidate join the same in-app call from separate networked computers. Each browser publishes its own microphone track to a WebRTC media room; a server-side analysis worker is intended to process the interviewer and candidate audio tracks independently, produce timestamped speech and analysis events, and deliver them to the interviewer dashboard. LiveKit is the recommended initial media-room option, while FastAPI remains the application/session backend. This target architecture is planned and is not part of the current local microphone-test prototype.

The initial product recommendation is audio-first. Whether the in-app call must include camera video is an open product decision; video/facial analytics remain out of scope. The system is designed to surface evidence and timestamps for human review.

1.1 Problem Statement

Manual review of interviews can make it difficult to track changes in speaking volume, pitch, pacing, pauses, answer relevance, and potentially rehearsed delivery. A structured analysis tool can help reviewers locate relevant moments and revisit the audio instead of relying only on memory.

1.2 Confirmed Project Decisions

Decision	Chosen direction
AI/script concern	Flag possible script-like delivery patterns only as review cues. Voice alone cannot establish AI authorship or intent.
Runtime	Live assistance during an interview conducted inside the web application.
Language	English for the MVP.
Question source	Detect questions from the live interviewer transcript; do not require questions to be entered in advance.
Audio inputs	Each participant uses their own microphone from their own networked computer; keep audio tracks separate.
Deployment assumption	Two participant browser clients join one authenticated interview session over a network.
In-app call media	Use WebRTC with an SFU; LiveKit is the recommended initial option, subject to a small prototype and service review.
Server-side analysis	An authorized worker subscribes to the original

Decision	Chosen direction
	interviewer and candidate audio tracks separately. The dashboard WebSocket carries structured events, not call media.
Video scope	Audio-only MVP recommended. Whether the call UI must include camera video is unresolved; video/facial analysis is out of scope.

2. Goals, Scope, and Non-Goals

2.1 Goals

- Capture one participant microphone on each of two networked computers and bind the published track to an authenticated interviewer or candidate role.
- Process interviewer and candidate streams independently while maintaining a shared interview timeline.
- Detect speech segments, pauses, and turn boundaries with timestamps.
- Transcribe English speech and identify interviewer questions from the recorded conversation.
- Link candidate answers to the most likely preceding question and analyse semantic relevance.
- Measure candidate loudness/energy, pitch, speaking rate, pitch variation, and pause duration over time.
- Flag possible script-like delivery patterns for human review with evidence and timestamps.
- Provide an understandable live dashboard and a final review report.

2.2 In Scope for the Initial Product

- In-app WebRTC audio calling between interviewer and candidate, with authenticated participant roles.
- English speech-to-text transcription.
- Voice activity detection and segmentation.
- Candidate voice measurements and change detection relative to the candidate baseline.
- Automatic question detection and question-answer association.
- Question-answer relevance flags.
- Interpretable scripted-delivery indicators (not proof of script use).
- A lightweight web dashboard for monitoring and review.

2.3 Out of Scope for the Initial Product

- Video/facial analysis and emotion recognition from video. Whether the call UI must include camera video remains unconfirmed; an audio-only first release is recommended.
- Automatic candidate rejection or ranking based on voice features.
- Definitive detection that a human-spoken answer was authored by an AI tool.
- Synthetic voice/deepfake detection unless the team lead adds it as a separate requirement.
- Support for languages other than English in the first release.
- A production multi-tenant deployment, analytics warehouse, or long-term recording archive.

3. Requirements and Acceptance Criteria

3.1 Functional Requirements

ID	Requirement	Acceptance criterion
FR-01	The interviewer and candidate each grant microphone permission on their own computer.	Each browser can publish its own microphone track to the session after permission is granted.
FR-02	The system assigns participant roles through authenticated session membership and role-specific join authorization.	A user cannot join as interviewer or candidate merely by changing a client-side role field.
FR-03	The application carries and processes two participant audio tracks independently.	The server-side analysis path receives the correct source track for each role.
FR-04	The system transports audio frames to the backend in a live session.	Each message includes a session identifier, role, sequence/timestamp metadata, and audio payload.
FR-05	The system identifies speech and non-speech intervals.	Segments include start/end timestamps and a role.
FR-06	The system transcribes English speech.	Partial and/or final transcript events are displayed with timestamps where supported.
FR-07	The system detects interviewer questions from live speech.	Question candidates are created from interviewer segments, including indirect questions when possible.
FR-08	The system associates candidate answers with questions.	Each answer stores a question reference or an explicit unresolved status.
FR-09	The system measures candidate voice characteristics over time.	The UI displays timestamped energy, pitch, speech rate, and pause-related data where estimable.
FR-10	The system evaluates answer relevance.	A flag contains the question, answer excerpt, rationale, and confidence/uncertainty.
FR-11	The system flags script-like delivery patterns.	A flag lists measurable features and a timestamp; it does not assert proof of reading or AI authorship.
FR-12	The reviewer can navigate to a timestamp and replay the segment, if audio retention	A selected flag opens the relevant audio interval.

ID	Requirement	Acceptance criterion
	is enabled.	
FR-13	The application can end the call/session and release media resources.	Participant tracks stop, users receive session-ended status, and browser media resources are released.
FR-14	The system produces a final interview review summary.	The report includes segments, flags, and limitations without automatic pass/fail decisions.
FR-15	The application creates an in-app interview room and supports joining from two networked computers.	Both participants can join the same room and leave it cleanly.
FR-16	The application uses WebRTC media transport through the selected SFU and exposes separate audio tracks to authorized analysis processing.	The candidate and interviewer tracks remain separate and role-labelled throughout processing.
FR-17	The backend issues short-lived role-specific media access tokens after authenticating the participant.	Expired, invalid, or wrong-role token attempts are rejected.
FR-18	The dashboard receives structured events over a separate application WebSocket.	Connection status, transcript, metrics, and review flags update without carrying call audio over the event channel.
FR-19	The call handles mute, disconnect, reconnect, and participant departure.	The UI exposes state changes, and the processing pipeline does not silently mislabel resumed tracks.

3.2 Non-Functional Requirements

ID	Requirement
NFR-01 Latency	The dashboard should update during the live interview. Establish a target after measuring transcription, network, and analysis latency on target hardware.
NFR-02 Reliability	Device disconnects, permission denial, API failures, and network interruptions must be surfaced clearly.
NFR-03 Security	API keys remain server-side; connections use HTTPS/WSS outside localhost; only authorized users can access sessions.

ID	Requirement
NFR-04 Privacy	Obtain informed consent, minimize retention, restrict access, and provide a defined deletion process.
NFR-05 Explainability	Each flag must show the observable signal or text evidence that triggered it.
NFR-06 Fairness	Evaluate with multiple English accents, speaking styles, microphones, noise levels, and accessibility needs.
NFR-07 Maintainability	Keep capture, transport, transcription, voice analytics, relevance analysis, and UI as separate modules.
NFR-08 Cost control	Track third-party audio/model use, limit session duration where needed, and avoid sending duplicate audio for overlapping processing.

3.3 Release Acceptance Criteria

- The interviewer and candidate can join an in-app call from separate networked computers and grant microphone permission.
- Authenticated join flows map each participant to the correct role and prevent a participant from joining with an unauthorized role.
- Audio from each role remains separate in the backend and is timestamped.
- The live transcript labels the source role correctly for both inputs.
- A completed interviewer question can be linked to a candidate response or marked unresolved.
- Voice measurements include timestamps and are compared to the candidate baseline rather than raw cross-microphone levels.
- Any script-like or relevance flag includes an explanation and is reviewable by a human.
- A test suite covers microphone permission denial, wrong-role attempts, audio-track separation, network interruption/reconnection, silence, overlap, participant departure, and session stop.

4. System Architecture and Data Flow

The updated target architecture separates the in-app call media plane from the application control and analysis planes. Browser clients use WebRTC to join a shared media room. The media service forwards the call and makes individual participant tracks available to an authorized server-side analysis worker. FastAPI handles application sessions, role authorization, join tokens, structured dashboard events, and reports. Analysis events remain timestamped and role-labelled on a shared interview timeline.

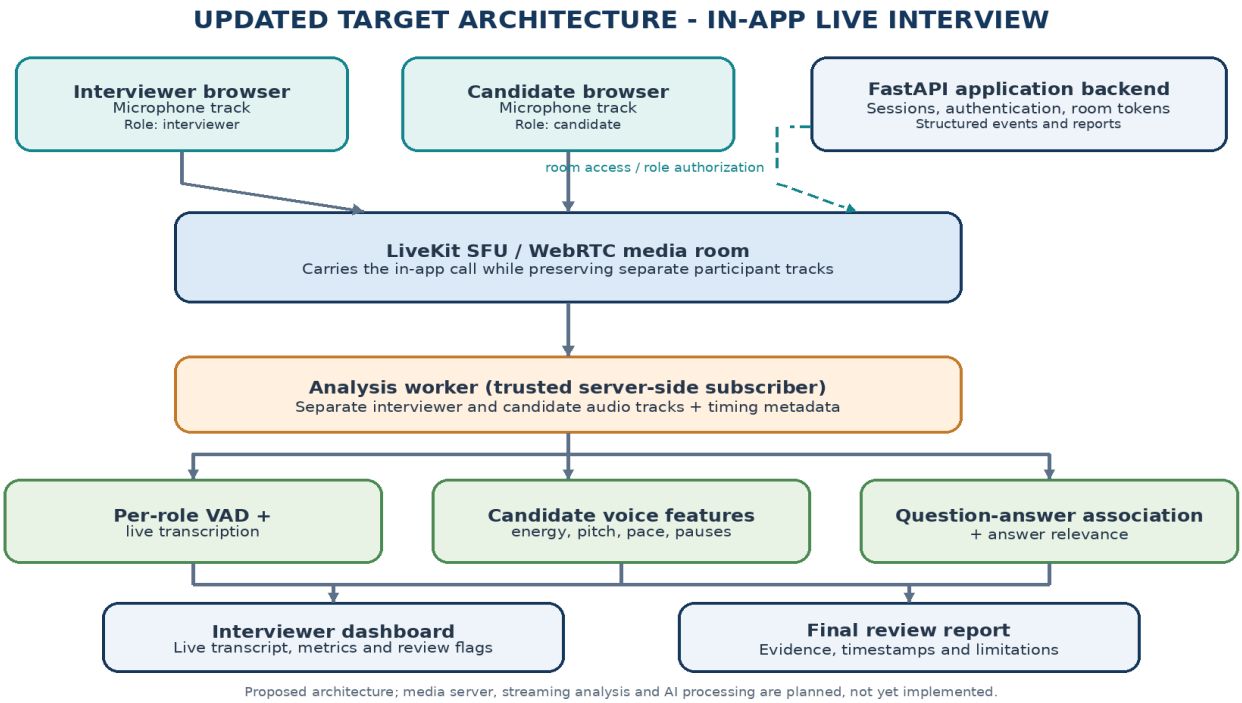


Figure 1. Updated target architecture for an in-app call across two networked computers. LiveKit/WebRTC, the analysis worker, transcription, and AI analysis are planned components; the current starter code only tests local microphone capture.

4.1 End-to-End Data Flow

1. The interviewer and candidate open the web application on separate computers, authenticate, and join the same interview session using role-specific access.
2. Each browser requests microphone permission and publishes its own microphone track to the WebRTC media room.
3. The media service carries the call. An authorized analysis worker subscribes to the interviewer and candidate tracks separately and preserves participant identity, track identifiers, timing metadata, and audio-format information. Do not use the application dashboard WebSocket as the primary transport for call audio.
4. The analysis worker adapts each track to the formats required by voice activity detection, streaming transcription, and voice-feature extraction. FastAPI separately delivers structured status and analysis events to the dashboard over WSS.
5. The interviewer transcript is analysed for question candidates. Candidate transcript segments are associated with a preceding question.
6. Candidate audio is analysed for energy/loudness, pitch, pauses, pace, and changes relative to the candidate baseline.
7. Relevance and script-like indicators are generated after enough context is available. Provisional flags may be updated as a speaking turn finishes.
8. The dashboard receives transcript, metrics, status updates, and review flags; the report summarizes the interview with evidence.

4.2 Timing and Synchronization

The participants use different computers, so their local clocks must not be assumed to match. Preserve per-track media timestamps where available, record server-side receive times, and align results to a session

timeline using explicit timing metadata. Handle track publication changes, reconnects, mute states, and gaps. Do not claim sample-accurate synchronization unless it has been measured and demonstrated.

4.3 Audio Format Strategy

Choose and document one internal audio format early, for example mono PCM at a supported sample rate for analysis. If a transcription service requires a different encoding or sample rate, convert explicitly in a dedicated audio adapter. Validate frame size, byte order, channels, and sample rate; never treat arbitrary byte chunks as a valid audio file without framing metadata.

4.4 In-App Call Architecture (Updated Target)

The call itself runs inside the web application. The interviewer and candidate join from separate networked computers. Each browser publishes its own microphone track using WebRTC. LiveKit is the recommended initial SFU/media-room option because the analysis worker must obtain each participant track separately; the choice remains subject to a small technical spike, service terms, region/data-handling review, and cost assessment. This is a target design, not an implemented feature.

Media plane: LiveKit/WebRTC carries the live conversation and forwards the original participant tracks. A server-side worker subscribes to the interviewer and candidate tracks independently, preserving authenticated participant identity, track ID, timestamps, mute state, and reconnect information. Use a supported server-side track subscription or per-track streaming/egress path.

Application control plane: FastAPI creates interview sessions, authenticates users, authorizes the interviewer/candidate roles, issues short-lived room tokens, ends sessions, and serves reports. A separate WebSocket over WSS sends structured dashboard events (transcripts, metrics, flags and status); it is not the primary transport for live call audio.

Recommended MVP scope: audio-only calling is the simplest first milestone. Whether camera video is required for the call remains an open product decision. If video is later added, it can use the same media room, but video/facial/behavioral analytics remain out of scope unless separately approved.

Connectivity and trust: serve the app over HTTPS and use WSS for application events. Test WebRTC on home and corporate networks and provide TURN support where required. Because the analysis worker must inspect audio, it is a trusted media consumer; a strict end-to-end encryption design that prevents server-side access would conflict with this processing model. Obtain participant consent and document retention.

5. Functional Modules

5.1 Browser Capture

Responsibilities: request microphone permission, create the local microphone track, display mute/connection state, publish the track through the LiveKit JavaScript SDK, and release media resources when leaving the call. Browser APIs include getUserMedia() and MediaStream. Authenticated session membership determines whether the participant is the interviewer or candidate; the target app does not ask one browser to select microphones for both remote people. The current prototype only implements local microphone testing.

5.2 Real-Time Media Transport and Session Events

Responsibilities: use WebRTC/LiveKit for the live call, create and authorize rooms, issue short-lived role-specific join tokens, and expose individual participant audio tracks to an authorized analysis worker. Use a separate WebSocket over WSS for structured dashboard events such as transcript updates, metrics, flags, and

processing status. Raw call audio is not intended to be sent through the dashboard event WebSocket. This module is planned and is not in the current starter code.

5.3 Voice Activity Detection and Segmentation

Responsibilities: distinguish speech from silence/noise and emit timestamped intervals. Segmentation should not assume each pause marks a new sentence; use silence thresholds, end-of-turn logic, and optional transcription events.

5.4 Speech-to-Text

Responsibilities: produce partial/final English transcripts and timestamps where available. Maintain separate transcript streams per role. Keep provider credentials on the server and handle rate limits, provider errors, and disconnections.

5.5 Question Detection and Answer Pairing

Responsibilities: classify interviewer utterances as question candidates, handle compound or follow-up questions, and match each candidate answer to the appropriate question. Store the association method and allow unresolved/ambiguous cases rather than forcing a wrong match.

5.6 Voice Feature Extraction

Responsibilities: compute short-window RMS energy, pitch/F0 when voiced, pitch range/variation, speech rate, pause durations, and changes from the candidate baseline. Ignore unvoiced/noisy windows for F0. Treat energy as a relative signal unless microphone calibration is available.

5.7 Answer Relevance Analysis

Responsibilities: compare the candidate response to the interviewer question using semantic similarity plus a question-specific rubric/model. Return a rationale and relevant text evidence. A low score is a review flag, not a definitive judgement that the candidate avoided a question.

5.8 Script-Like Delivery Analysis

Responsibilities: combine multiple interpretable signals across sufficiently long candidate segments, such as unusually regular pacing or low intonation variation. Compare within the same candidate, not across candidates. If a reference script is explicitly provided later, text similarity can be a separate stronger check. Do not infer AI authorship from delivery style alone.

5.9 Dashboard and Reporting

Responsibilities: display call/connection status, participant roles, transcript, audio timeline, voice graphs, review queue, and final report. Every alert should open the corresponding text/audio segment when retention permits.

6. Technology Stack

Layer	Preferred starting option	Notes
Frontend	React + TypeScript, or HTML/CSS/JavaScript for the first prototype	Build the call UI, join flow, participant status, and interviewer dashboard.
Browser audio	MediaDevices + Web Audio API	Each browser requests permission for its participant microphone and publishes a WebRTC track; HTTPS is required outside localhost.
Backend	Python + FastAPI	FastAPI manages sessions, authentication/authorization, role-specific room tokens, dashboard events, and reports.
Audio analysis	NumPy, SoundFile, librosa	Use a separate worker subscribed to individual participant tracks; normalize/resample explicitly where needed.
Voice activity detection	A suitable VAD implementation	Evaluate latency, robustness to noise, and deployment requirements before selection.
Transcription	A live speech-to-text API or self-hosted model	Compare English accuracy, streaming support, latency, costs, timestamp quality, and privacy.
Question-answer analysis	Embeddings plus a structured LLM/classifier rubric	Return structured JSON with rationale, uncertainty, and evidence excerpts.
Dashboard charts	Chart.js or another browser chart library	Render timestamp-aligned voice features and flags.
Storage	In-memory for early prototype; SQLite/PostgreSQL later	Persist only what is needed. Store recordings only under explicit consent, retention, and access-control policy.
Media transport	LiveKit WebRTC SFU (recommended initial option)	Carries the in-app call and forwards separate participant tracks. Evaluate LiveKit Cloud first; self-host only if justified by operations, cost, or data-residency needs.

Layer	Preferred starting option	Notes
Server-side media integration	LiveKit server-side SDK or per-track egress	The worker subscribes to role-labelled tracks or receives track audio through a supported per-track streaming path.
Dashboard event transport	FastAPI WebSocket over WSS	Carry structured JSON events such as transcript.partial, voice_metrics, question.detected, answer.analysis, review.flag, and session_status; not the call media itself.

Technology selection remains subject to a prototype spike and stakeholder review. LiveKit Cloud is the recommended starting media service because the target requires a call inside the web app and server access to separate participant tracks; self-hosting is an option if operational capability, cost, or data-residency requirements justify it. Before selection, verify current pricing, region/data handling, track-subscription and egress behavior, supported formats, latency, licensing, and terms. Keep transcription-provider credentials server-side. The local microphone prototype can still be tested without external API keys.

7. Repository Structure and Setup

7.1 Suggested Repository Structure

```
voice-interview-analyzer/
├── .venv/                  # Local virtual environment (not committed)
├── main.py                 # FastAPI application entry point
├── requirements.txt        # Python dependencies (to add)
├── .env.example            # Placeholder configuration (never commit secrets)
├── .gitignore
├── static/                 # Current local prototype UI
│   └── index.html          # Local microphone test
├── frontend/               # Planned in-app call UI and dashboard
│   ├── index.html
│   ├── css/
│   └── js/
├── app/
│   ├── api/                # REST and dashboard WebSocket endpoints (planned)
│   ├── call/               # LiveKit rooms, join tokens, role authorization
│   ├── audio/              # Audio normalization, timing, format adapters
│   ├── workers/            # Server-side participant-track analysis (planned)
│   ├── transcription/      # Provider adapter / transcription (planned)
│   ├── segmentation/       # VAD and segment logic (planned)
│   ├── analysis/           # Voice metrics, relevance, review flags
│   ├── schemas/            # Pydantic event and result schemas
│   └── services/           # Session lifecycle and report services
├── tests/
│   ├── test_health.py
│   ├── test_audio_metadata.py
│   └── test_analysis.py
└── README.md
```

7.2 Prerequisites

- Windows 10/11 or another supported development OS.
- Python 3.11 or another version supported by selected dependencies.
- VS Code or another editor.
- Two networked computers with a modern browser and one microphone per participant; HTTPS-accessible app hosting and a configured LiveKit environment are needed for the target call.
- Network access is required for the in-app call/media service; internet access is also required when hosted transcription or analysis providers are selected.

7.3 Windows PowerShell Setup

```
mkdir voice-interview-analyzer
cd voice-interview-analyzer
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install fastapi "uvicorn[standard]"
mkdir static
```

Create main.py and static/index.html using the starter code supplied with this documentation. If Python is not found under the `python` command, try `py -m venv .venv`. If PowerShell blocks virtual-environment activation, follow your organization-approved execution-policy approach or activate the environment through a supported terminal.

7.4 Run the Local Prototype

```
uvicorn main:app --reload
```

For the local starter only, open http://127.0.0.1:8000 and check /health, then load microphones, grant access, test the meter, and stop capture. This localhost workflow does not deliver a call between two computers. The target in-app call requires a publicly/routably reachable application and media service, HTTPS for browser microphone access, WSS for structured events, valid LiveKit configuration, and role-specific join authorization.

7.5 Current Backend Endpoints

Method and path	Status	Purpose
GET /	Implemented	Returns the current starter static page.
GET /health	Implemented	Returns a basic backend health response.
POST /api/sessions	Planned	Creates an interview session and associated room/session state.
POST /api/sessions/{id}/join-token	Planned	Issues a short-lived role-specific LiveKit access token after authorization.
WS /api/sessions/{id}/events	Planned	Streams structured dashboard events; this is not the primary

Method and path	Status	Purpose
		call-audio transport.
POST /api/sessions/{id}/finish	Planned	Ends the session, stops analysis, and finalizes results.
GET /api/sessions/{id}/report	Planned	Returns the structured review report.

8. Initial Prototype: How It Works

The current starter implementation is a local microphone test, not the in-app calling or AI analysis product. It contains a minimal FastAPI server and a single HTML page with JavaScript. It does not establish a two-person WebRTC room or provide server-side participant-track processing.

8.1 main.py

- Creates the FastAPI application.
- Mounts the static directory.
- Serves index.html at the root URL.
- Exposes /health for a basic backend health check.

8.2 static/index.html

- Requests microphone permission so the browser can reveal audio device labels.
- Uses enumerateDevices() to list audio inputs.
- Lets the operator choose different devices for interviewer and candidate roles.
- Requests each selected device separately using getUserMedia().
- Creates an AudioContext and AnalyserNode for each stream.
- Calculates a simple waveform RMS-like value and displays a moving relative-level meter.
- Stops the animation loop, tracks, and AudioContext when the test ends.

8.3 What the Prototype Does Not Yet Do

- It does not send audio to FastAPI or a WebSocket.
- It does not record or save interview audio.
- It does not transcribe speech or detect questions.
- It does not calculate pitch, speech rate, or pause metrics in a persisted timeline.
- It does not run question relevance or script-like-delivery models.
- It does not persist sessions, generate a report, or provide authentication.

8.4 Known Prototype Caveat

The two-device caveat above applies to the legacy local microphone-test prototype. In the target application, each computer captures its own participant microphone. Role assignment comes from authenticated session membership and server-issued room permissions, not from selecting both microphones in one browser. Validate browser microphone permissions and WebRTC connectivity on the actual target networks. Some networks require TURN relay support; connection handling and fallback behavior must be tested.

9. Planned API and Event Contracts

The following interfaces are proposed application contracts, not implemented endpoints. LiveKit/WebRTC carries the live media. FastAPI creates sessions and role-specific join tokens, while an application WebSocket sends structured analysis events to the dashboard.

9.1 Session Creation (planned)

```
POST /api/sessions
Request:
{
  "language": "en",
  "roles": ["interviewer", "candidate"],
  "audio_mode": "in_app_webrtc"
}

Response:
{
"session_id": "generated-session-id",
  "status": "created"
}
```

9.2 Media Track Identity and Audio Processing (planned)

The browser publishes audio as a WebRTC track. The analysis worker receives decoded frames through a server-side track subscription or a per-track WebSocket egress path. Metadata below illustrates what the worker should associate with a track; it is not a requirement for the browser to push raw PCM chunks to FastAPI.

```
{
"source": "participant_track",
"session_id": "generated-session-id",
  "role": "candidate",
"track_id": "media-track-id",
"server_received_ms": 168000,
"sample_rate_hz": 48000,
"channels": 1,
"encoding": "pcm_s16le"
}
```

9.3 Server Events (planned)

Event type	Key fields	Purpose
session_status	session_id, status, message	Connection and processing state.
transcript.partial	role, text, start_ms, end_ms, is_final	Live transcript updates.

Event type	Key fields	Purpose
segment.completed	role, segment_id, start_ms, end_ms	Completed speech/turn interval.
voice_metrics	role, start_ms, end_ms, rms, f0_hz, speech_rate	Timestamped candidate measurements; fields may be null if unavailable.
question.detected	question_id, start_ms, end_ms, text, confidence	Question candidate from interviewer stream.
answer.analysis	answer_id, question_id, relevance, rationale, evidence	Candidate answer relevance result.
review.flag	flag_id, category, start_ms, end_ms, explanation, uncertainty	Human-review flag.
error	code, message, recoverable	Actionable error information.

9.4 Example Review Flag

```
{
  "flag_id": "flag-012",
  "category": "voice_energy_change",
  "role": "candidate",
  "start_ms": 261000,
  "end_ms": 269000,
  "explanation": "Speech energy increased relative to this candidate's baseline.",
  "evidence": {"baseline_rms": 0.03, "segment_rms": 0.05},
  "uncertainty": "Microphone gain and distance may affect this measurement.",
  "requires_human_review": true
}
```

10. Testing and Evaluation Plan

10.1 Prototype Tests

Test	Procedure	Expected result
Permission granted	Click Load Microphones and allow access.	Microphone inputs are listed.
Permission denied	Block microphone permission and reload/test.	Readable error appears; app does not crash.
Two distinct inputs	Select two separate device IDs and start.	Both streams open; both meters respond.
Role mapping	Speak only into interviewer input, then only candidate input.	Only the matching meter reacts prominently.
One device missing	Disconnect one microphone and reload devices.	Start is disabled or user is told to connect a second device.

Test	Procedure	Expected result
Device removed during test	Unplug a microphone while testing.	System shows a failure state; other resources can be stopped safely.
Stop test	Click Stop Test.	Tracks stop, meters reset, and audio context is closed.

10.2 Full-System Tests

- In-app calling: interviewer and candidate join from separate computers, can hear one another, and can mute/leave; verify role-specific tokens and reject unauthorized role changes.
- Media transport and track handling: test dropped or reordered media, track mute/unmute, participant departure, reconnection, server-side subscriber failures, and timing gaps.
- Segmentation: long silence, short pauses, false speech detections from noise, interruptions, and overlapping voices.
- Transcription: different English accents, fast speech, technical terms, disfluencies, quiet speech, and noisy rooms.
- Question detection: direct questions, indirect questions, compound questions, follow-ups, and statements that contain question-like words.
- Answer pairing: one answer addressing multiple questions, candidate asking a clarifying question, interruption before answer, and no answer.
- Voice metrics: establish a baseline and verify changes with controlled gain, microphone distance, background noise, and head movement.
- Relevance analysis: include relevant paraphrases, partially relevant answers, off-topic answers, and valid clarifying responses.
- Script-like flags: test read speech, memorized speech, natural structured answers, and different speaking styles; record false-positive and false-negative rates.
- Security: verify API keys are not sent to the browser, sessions cannot be opened by unauthorized users, and recordings are not retained unexpectedly.

10.3 Evaluation Metrics

Component	Suggested metric
Transcription	Word error rate (WER), latency, and proportion of segments with missing/incorrect timestamps.
Speech segmentation	Boundary error and missed/false speech duration.
Question detection	Precision, recall, and F1 on a manually labelled set of interviewer turns.
Question-answer association	Exact pair accuracy and unresolved-case rate.
Relevance	Agreement with human-labelled relevant / partially relevant / off-topic examples; precision and recall by label.
Voice measurements	Agreement with controlled test changes and stability under

Component	Suggested metric
	microphone/noise variation.
Review flags	False-positive rate, reviewer usefulness, and calibration of confidence/uncertainty.
End-to-end system	End-to-end latency, dropped frames, uptime during a test session, and resource use.

Do not select arbitrary success thresholds without a representative dataset and stakeholder agreement. Collect labelled, consented recordings and choose metrics and thresholds before trialing the system in a real hiring workflow.

11. Security, Privacy, and Responsible Use

- Consent and notice: tell participants that audio is processed, what features are analysed, whether a third-party service is used, and how long data is retained.
- Data minimization: retain only the transcript, metrics, and review markers needed for the stated purpose. Store raw recordings only if necessary and approved.
- Secret management: API keys, LiveKit API secrets, and media-room credentials stay server-side. Issue short-lived role-specific join tokens only after authorization. Never put provider keys in HTML/JavaScript or commit them to Git.
- Transport security: use HTTPS and WSS outside local development. Authenticate and authorize access to interview sessions and reports.
- Access controls: restrict session creation, live monitoring, report access, exports, and deletion actions to authorized users.
- Retention/deletion: define automatic expiry and a way to delete raw audio, transcripts, metrics, and derived reports as a coherent set.
- Human oversight: flags should direct a reviewer to evidence, not make hiring decisions. A voice change is not proof of stress, dishonesty, deception, or poor capability.
- Fairness: accents, speech impairments, neurodiversity, culture, room acoustics, and microphone quality can influence measured signals. Test the system across varied conditions.
- Auditability: record model/provider versions, configuration, and when flags were generated, without logging secrets or unnecessary sensitive data.

12. Known Limitations and Risks

Risk / limitation	Impact	Mitigation
AI authorship cannot be reliably inferred from human voice alone.	A candidate who speaks an AI-written answer may sound like anyone else giving a prepared answer.	Report observable delivery patterns and answer relevance separately; do not label content AI-written without separate evidence.
Script-like speech is not proof of reading.	Confident, rehearsed, neurodivergent, or naturally steady speakers may be flagged.	Use multiple weak indicators, within-speaker baselines, human review, and false-positive testing.

Risk / limitation	Impact	Mitigation
Microphone gain differs by device.	Raw signal amplitude between devices is not directly comparable.	Use candidate-only baseline and calibrate levels; avoid absolute loudness claims unless calibrated.
Automatic gain control / noise suppression.	Processing can mask or create apparent amplitude changes.	Test browser and hardware settings; document the input pipeline.
Question pairing can be ambiguous.	Follow-ups, compound questions, and interruptions can lead to incorrect matches.	Allow multiple question references, unresolved state, and reviewer corrections.
Transcription errors.	A wrong transcript can cause false context flags.	Display transcript uncertainty where available and make text/audio review easy.
Latency and provider outage.	Dashboard flags may arrive late or be missing.	Use provisional flags, robust retry/error states, and clearly label incomplete analysis.
Privacy exposure.	Interview audio is sensitive personal data.	Consent, encryption, restricted access, minimum retention, and deletion controls.
WebRTC connectivity and network restrictions.	Corporate firewalls, NAT behavior, or poor networks may prevent or degrade the in-app call and can delay analysis.	Test on representative networks, provide TURN relay support where needed, expose connection state, and test recovery after reconnect.

13. Development Roadmap

Milestone	Deliverable	Exit condition
M0 - Local capture prototype	Keep the current FastAPI home/health endpoints and microphone meter test as a baseline.	Permission, meter, and stop behavior work reliably on the development computer.
M1 - In-app two-person call	WebRTC room using LiveKit, role-specific join tokens, call UI, mute/leave states.	Interviewer and candidate join from separate computers and can hear one another.
M2 - Separate server-side tracks	Analysis worker subscribes to interviewer and candidate tracks; handle participant join/leave and reconnect.	Each track is available independently with correct role mapping and usable timing metadata.
M3 - Live transcription and segmentation	Per-role partial/final transcripts, VAD boundaries, and timestamps.	Both transcript streams update live and remain associated with

Milestone	Deliverable	Exit condition
		the correct participant.
M4 - Candidate voice measurements	RMS energy, pitch/F0 estimates, pace, pauses, and within-candidate baseline changes.	Timestamped charts are stable in repeatable tests and do not compare raw amplitudes across devices.
M5 - Question-answer understanding	Question candidates, candidate answer boundaries, pairing, relevance rubric, explanations.	Reviewer can inspect the question, answer, rationale, and evidence; ambiguous cases remain unresolved.
M6 - Review flags and production hardening	Script-like delivery observations, report, authentication, retention, monitoring, and tests.	Flags remain non-conclusive; security, privacy, and fairness review is complete.

13.1 Recommended Immediate Next Step

Keep the existing local microphone test as a baseline, then implement M1: an in-app two-person audio call with LiveKit/WebRTC, secure room creation, and role-specific join tokens. Next, add a server-side worker that receives the interviewer and candidate tracks independently and verifies role mapping and timing. Add transcription only after this media path is reliable. The camera/video requirement is still to be confirmed; an audio-only MVP is recommended unless video is a hard requirement.

14. Glossary and References

Term	Meaning
VAD	Voice Activity Detection: estimating which audio intervals contain speech.
Segmentation	Dividing audio into timestamped intervals or speech turns.
RMS energy	A measure of signal amplitude over a window; related to signal level, but not automatically a calibrated measure of perceived loudness.
F0 / fundamental frequency	The rate of vocal fold vibration in voiced speech; often perceived as pitch.
WebSocket	A persistent bidirectional connection used here for structured application events; WebRTC/LiveKit carries the live call media.
Question-answer association	Mapping each interviewer question to one or more candidate

Term	Meaning
	response segments.
Script-like indicator	A measurable delivery pattern that may warrant review; not a definitive statement about reading or AI use.
WER	Word Error Rate, a common measure for comparing a transcription to a reference transcript.
WebRTC	Browser technology used to establish real-time audio/video media communication.
SFU	Selective Forwarding Unit: media server that forwards participant tracks to authorized subscribers without requiring a mixed call recording.
LiveKit	Recommended initial WebRTC media-room platform for the in-app call; integration is planned, not implemented.
Analysis worker	Authorized server-side process that subscribes to individual participant audio tracks and runs audio/transcript analysis.
TURN	Traversal Using Relays around NAT: relay infrastructure used when direct WebRTC connectivity is not possible.

Technical References

- MDN - MediaDevices.getUserMedia(): https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
- MDN - MediaDevices.enumerateDevices(): https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/enumerateDevices
- FastAPI - WebSockets: https://fastapi.tiangolo.com/advanced/websockets/
- OpenAI API - Realtime transcription guide: https://developers.openai.com/api/docs/guides/realtime-transcription
- librosa - pYIN pitch estimation: https://librosa.org/doc/latest/generated/librosa.pyin.html
- Chart.js documentation: https://www.chartjs.org/docs/latest/
- LiveKit - Processing raw media tracks: https://docs.livekit.io/transport/media/raw-tracks/
- LiveKit - Track egress and audio WebSocket streaming: https://docs.livekit.io/transport/media/ingress-egress/egress/track/
- LiveKit - Publishing camera and microphone tracks: https://docs.livekit.io/transport/media/publish/
- MDN - WebRTC connectivity: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Connectivity

Appendix A. Minimal Backend Skeleton

The following shows the backend structure currently used in the prototype. It only serves the web page and health endpoint. The streaming and analysis APIs listed in Section 9 must be added later.

```
from pathlib import Path

from fastapi import FastAPI
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

BASE_DIR = Path(__file__).resolve().parent
STATIC_DIR = BASE_DIR / "static"

app = FastAPI(title="Voice Interview Analyzer")
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

@app.get("/")
def home():
    return FileResponse(STATIC_DIR / "index.html")

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Appendix B. Environment and Secret Handling

If external services are integrated, create a local .env file only for development and add .env to .gitignore. Commit an .env.example with placeholder names only. Never commit real keys, tokens, interview audio, or candidate transcripts to source control.

```
# .env.example (placeholders only)
TRANSCRIPTION_PROVIDER=replace_me
TRANSCRIPTION_API_KEY=replace_me
AUDIO_RETENTION_HOURS=0
LOG_LEVEL=INFO
```

Appendix C. Definition of “AI Detection” for This Project

For this MVP, AI-related analysis means (a) possible script-like delivery signals in a human voice and (b) whether the spoken answer is relevant to the detected question. These signals cannot determine reliably whether ChatGPT or another AI tool authored an answer. Detecting synthetic voice audio would be a separate feature, outside the initial scope. All results must be phrased as observations or review flags, not conclusions about intent.